

ODA-Bench: Benchmark offline từ real-log cho đánh giá tránh vật cản UAV

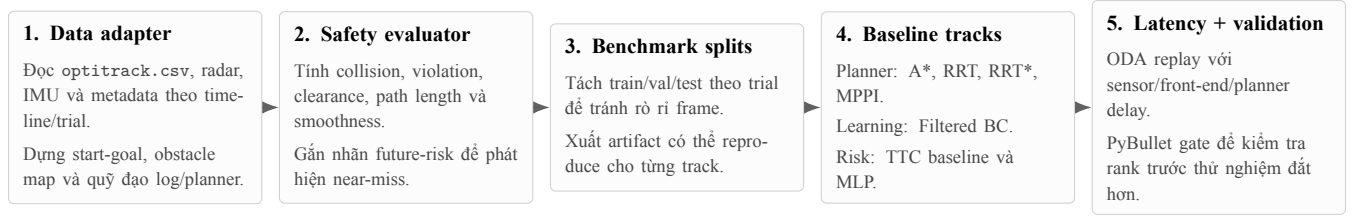
Nguyễn Thanh Lâm
Sinh viên

Hoàng Công Phát
Mentor
Viettel High Tech

Tóm tắt

Đề tài xây dựng **ODA-Bench**, một benchmark offline dùng log UAV thật từ ODA Dataset để đánh giá tránh vật cản trước khi đưa ứng viên sang mô phỏng hoặc thử nghiệm thực địa. Thay vì tạo dataset mới hoặc claim một planner tự hành mới, đóng góp chính là chuẩn hóa ODA raw logs thành protocol có thể lặp lại: đọc OptiTrack/radar/IMU/metadata, dựng hình học vật cản, tạo nhãn collision/violation/clearance, tách train/val/test theo trial, chạy các baseline planner, Filtered Behavior Cloning (Filtered BC), risk predictor, latency-aware ODA replay và kiểm chứng lại bằng PyBullet validation. Kết quả hiện tại gồm 300 ODA trials, 14,700 policy-dataset samples, planner/risk/BC tracks, sensor/front-end latency, ODA replay online feasibility 384 cases và PyBullet validation 20 cases/method; các số liệu chính cho thấy benchmark tách được unsafe path, near-miss policy, risk ranking tốt hơn TTC baseline và giới hạn tốc độ do latency cảm biến/tính toán. Các planner dùng metadata obstacle map được trình bày như privileged-map sanity baseline, không phải bằng chứng perception-limited online autonomy.

Từ ODA dataset → task + metric + baseline → quyết định ứng viên rõ hơn



Đầu ra cuối: bảng metric, risk labels, video định tính, ODA replay và simulator validation để chọn cấu hình đáng thử tiếp.

Fig. 1. Luồng thí nghiệm offline benchmark trên ODA-Bench.

I. GIỚI THIỆU CHUNG

UAV bay trong nhà hoặc vùng GNSS-denied khó đánh giá an toàn vì va chạm gây hỏng thiết bị, còn mô phỏng thuần túy dễ gặp khoảng cách sim-to-real. ODA Dataset cung cấp log UAV thật với RGB, event camera, radar, IMU, OptiTrack ground truth và metadata vật cản [1, 2], nhưng bản raw dataset chưa tự trả lời được câu hỏi kỹ thuật: cấu hình planner/policy/risk model nào an toàn hơn trên cùng tập trial?

Các hướng liên quan có ba nhóm chính. Nhóm UAV datasets như ODA, Blackbird hoặc Mid-Air cung cấp log hoặc dữ liệu bay, nhưng thường dừng ở dữ liệu thô hoặc synthetic data. Nhóm simulation benchmarks như AvoidBench và Flightmare tạo môi trường lặp lại cho UAV [3, 4], nhưng phụ thuộc vào simulator và có sim-to-real gap. Nhóm learning/offline policy như DroNet, D4RL, TD3+BC hoặc Decision Transformer cung cấp policy hoặc protocol học offline [5, 6, 7, 8], nhưng không được thiết kế riêng cho real-log UAV obstacle avoidance kiểu ODA.

Vì vậy, mục tiêu của đề tài là lấp khoảng trống giữa raw ODA logs và một protocol đánh giá an toàn có thể lặp lại.

Sản phẩm mong muốn không phải một thuật toán planner mới, mà là lớp benchmark/evaluator giúp lọc ứng viên sớm bằng số liệu: collision, safety violation, clearance, compute time, risk ranking/false negative và validation score. Cách đọc đúng của báo cáo là: ODA-Bench là tầng offline screening và validation gate, trong đó PyBullet được dùng để kiểm tra closed-loop trước khi đưa ứng viên sang thử nghiệm đất hơn.

II. NỘI DUNG VÀ PHƯƠNG PHÁP

A. Chuẩn hóa ODA thành benchmark

Pipeline đọc `optitrack.csv`, `radar.csv`, `imu.csv` và metadata vật cản của 300 trial ODA. Mặt phẳng bay được lấy theo OptiTrack $x - z$; vật cản được mô hình hóa thành hình trụ bán kính $r = 0.20\text{ m}$. Với vị trí UAV $\mathbf{p}_i$ và vật cản $\mathbf{o}_j$, clearance tới biên vật cản tại thời điểm i là:

$$c_i = \min_j \|\mathbf{p}_i - \mathbf{o}_j\|_2 - r.$$

Một trial bị tính collision nếu minimum clearance âm; bị tính safety violation nếu UAV đi vào vùng margin an toàn. Ngoài

collision/violation, evaluator ghi lại minimum clearance, path length, smoothness và thời gian tính toán để so sánh nhiều ứng viên trên cùng điều kiện start/goal/obstacle.

Dữ liệu downstream được tách theo trial ID để giảm rò rỉ frame cùng trial giữa train/test. Tập hiện tại gồm 210 train trials, 45 validation trials và 45 test trials, tương ứng 10,290/2,205/2,205 policy-dataset samples. Một policy-dataset sample là một timestep state-action sau khi resample trajectory: state gồm pose footprint (x, z) , vector tương đối tới goal, vector tới vật cản gần nhất, vận tốc, clearance và số vật cản; action là displacement cục bộ $(\Delta x, \Delta z)$ tới waypoint kế tiếp. Các mẫu này dùng cho học offline, không được hiểu là expert tuyệt đối vì chính evaluator cho thấy một phần log có near-miss. Cách tách này cho phép dùng cùng ODA-Bench cho planner evaluation, behavior cloning và risk prediction.

B. Các track đánh giá

Track planner so sánh human trajectory, straight-line, A*, geometric bypass, RRT, RRT* và MPPI trên hình học ODA. Các planner hình học trong track này dùng obstacle map từ metadata, vì vậy được dùng làm upper-bound/sanity baseline để kiểm tra evaluator và trade-off hình học, không phải bài toán perception online hoàn chỉnh. Track trajectory-quality đo thêm smoothness/turn-angle proxy 3D trên các trial có đủ height. Track imitation learning huấn luyện Plain BC-MPPI và Filtered BC-MPPI từ policy-dataset trajectories sinh bởi pipeline planner/demo; Filtered BC chỉ học từ mẫu có clearance đủ tốt để giảm học theo near-miss behavior. Track risk prediction so sánh TTC/distance baseline với một MLP nhỏ, trong đó PR-AUC đo chất lượng ranking còn FNR phụ thuộc threshold vận hành. Track latency dùng ODA RGB/cached depth để đo sensor-to-occupancy delay và dùng ODA OptiTrack replay tại thời điểm $t + T_{\text{delay}}$ để kiểm tra planner còn an toàn khi lệnh đến muộn. Track validation dùng gym-pybullet-drones/PyBullet [9] để kiểm tra closed-loop trên 20 cases/method, speed=1.

C. Giải thích các độ đo sử dụng

Với mỗi trial, evaluator lấy chuỗi vị trí UAV $\mathbf{p}_1, \dots, \mathbf{p}_N$ và các vị trí vật cản $\mathbf{o}_j$. Minimum clearance của trial là:

$$c_{\min} = \min_i c_i, \quad c_i = \min_j \|\mathbf{p}_i - \mathbf{o}_j\|_2 - r.$$

Từ đó, *collision rate* là tỉ lệ trial có $c_{\min} < 0$, còn *safety violation rate* là tỉ lệ trial có $c_{\min} < s$, với safety margin $s = 0.50 m$. Hai độ đo này tách contact thật khỏi near-miss: collision đo va chạm với vật cản, còn violation đo việc bay quá gần dù chưa chạm.

Path length đo quãng đường quỹ đạo:

$$L = \sum_{i=2}^N \|\mathbf{p}_i - \mathbf{p}_{i-1}\|_2.$$

Smoothness được đo bằng thay đổi hướng trung bình giữa các đoạn liên tiếp. Gọi $\theta_i = \text{atan2}(z_i - z_{i-1}, x_i - x_{i-1})$, khi đó

smoothness càng nhỏ nghĩa là đường đi ít gấp khúc hơn:

$$S = \frac{1}{N-2} \sum_{i=2}^{N-1} |\text{wrap}(\theta_{i+1} - \theta_i)|.$$

Compute time là thời gian thuật toán sinh quỹ đạo hoặc action cho một trial; đây là số đo latency offline trên cùng môi trường chạy, không phải FPS bay thật.

Với các thí nghiệm latency, tổng trễ được mô hình hóa theo chuỗi sensor-to-command:

$$T_{\text{delay}} = T_{\text{sensor}} + T_{\text{frontend}} + T_{\text{planner}} + T_{\text{publish}}, \\ d_{\text{delay}} = v T_{\text{delay}}.$$

T_{sensor} là chu kỳ cảm biến, T_{frontend} là thời gian đổi dữ liệu cảm biến thành occupancy/costmap, T_{planner} là thời gian planner hoặc controller tính lệnh, và d_{delay} là quãng đường UAV có thể đi trước khi lệnh mới có hiệu lực. Với ODA replay, pose bị trễ được lấy bằng nội suy OptiTrack tại $t + T_{\text{delay}}$, sau đó planner replanning từ pose này và được chấm lại collision/violation/clearance.

Với risk prediction, TTC (Time-To-Collision) ước lượng thời gian còn lại trước khi chạm nếu UAV tiếp tục tiến gần vật cản theo trạng thái hiện tại:

$$\text{TTC} = \frac{\max(c_i, 0)}{\max(v_{\text{close}}, \epsilon)}, \quad v_{\text{close}} = -\frac{dc_i}{dt}.$$

Nếu UAV không tiến gần vật cản thì $v_{\text{close}} \leq 0$ và TTC được xem là rất lớn. TTC nhỏ nghĩa là rủi ro cao. MLP risk predictor được đánh giá bằng PR-AUC để đo ranking trên dữ liệu mất cân bằng, risk recall = $\text{TP}/(\text{TP} + \text{FN})$, và false negative rate = $\text{FN}/(\text{TP} + \text{FN})$. Trong bài toán an toàn, false negative rate thấp là quan trọng vì bỏ sót nguy cơ nguy hiểm hơn báo động giả; tuy nhiên FNR phải được so sánh tại cùng operating point hoặc cùng mức recall/precision, không tách rời threshold.

Với PyBullet validation, ba outcome success/violation/collision được tính như ba nhóm loại trừ nhau và cộng thành 100%. Rank transfer đo mức offline ranking có giữ được thứ tự khi sang simulator hay không. Với n phương pháp, Spearman ρ được tính từ chênh lệch rank d_k :

$$\rho = 1 - \frac{6 \sum_{k=1}^n d_k^2}{n(n^2 - 1)}.$$

III. KẾT QUẢ THỰC HIỆN

Quy mô artifact chính của benchmark được tóm tắt trong Bảng 2; đây là nền cho các track planner, learning, latency và validation ở các phần sau.

A. Planner baseline trên 300 trials

Bảng 3 cho thấy benchmark phân biệt rõ unsafe baseline và các planner hình học khi obstacle map được lấy từ metadata ODA. Straight-line có collision 14.00% và violation 65.33%. Human trajectory trong log có collision 7.33% và violation 44.67%; đây là collision/violation theo mô hình evaluator với

Table 1. Môi trường đo tốc độ trong báo cáo.

Hạng mục	Cấu hình/diễn giải
Máy local	Apple M2, RAM 16GB, macOS 26.1; các script chạy bằng Python/NumPy trong workspace local.
Máy PyBullet validation	Windows PC, NVIDIA GTX 1050 Ti, Intel Core i5-7400, RAM 8GB. Đây là máy dùng cho kết quả PyBullet validation.
Setting PyBullet	Backend <code>gym-pybullet-drones/PyBullet</code> , GUI tắt, 20 seeds, speed = 1 m/s , altitude = 1.0 m , control frequency 48Hz, simulation frequency 240Hz.
Compute time planner	Thời gian tính quỹ đạo offline cho một trial; dùng để so sánh tương đối giữa planner trên cùng máy.
BC inference latency	Thời gian suy luận MLP nhỏ theo sample, không phải FPS camera hoặc tốc độ bay thật.
Nền tảng inference	Report hiện không claim so sánh FPS PyTorch/ONNX/TensorRT; nếu bổ sung sau cần gom vào một bảng riêng kèm cấu hình GPU/driver.

Table 2. Quy mô artifact chính của ODA-Bench hiện tại.

Hạng mục	Kết quả
ODA benchmark subset	300 trials có OptiTrack, radar, IMU, RGB/metadata; cùng protocol collision/violation/clearance.
Downstream split	210/45/45 train/val/test trials; 14,700 timestep state-action policy samples.
Baseline tracks	Planner, Filtered BC, risk predictor; offline RL giữ ở hướng mở rộng.
Latency tracks	ODA RGB/cached-depth sensor frontend, online delay model và ODA replay 384 cases từ OptiTrack.
Validation gate	PyBullet validation 20 cases/method, speed=1; so sánh RRT*, MPPI và Filtered BC.

Table 3. Planner safety trên 300 ODA trials.

Phương pháp	Trials	Collision	Violation	Clear. (m)	Compute (ms)
Human	300	0.0733	0.4467	0.6126	0.00
Straight-line	300	0.1400	0.6533	0.4203	2.87
A* grid	300	0.0000	0.0000	0.6426	8.68
Geom. bypass	196	0.0000	0.0051	0.7005	6.39
RRT	300	0.0000	0.0000	0.6702	3.62
RRT*	300	0.0000	0.0000	0.6360	1196.40
MPPI	300	0.0000	0.0033	0.7574	39.93

vật cản hình trụ và safety margin, không phải khẳng định toàn bộ các case đó là va chạm vật lý thật. A*, RRT và RRT* đạt 0% collision/violation trong benchmark 2D; MPPI đạt 0% collision, violation 0.33% và mean clearance cao nhất trong các planner chính 0.7574 m . Geometric bypass chỉ áp dụng cho các trial thỏa giả định simple-bypass/single-obstacle, nên được báo cáo trên 196 valid trials và không nên so sánh như một planner full-coverage. Vì các planner hình học dùng cùng safety inflation $r + s = 0.70\text{ m}$ như evaluator, kết quả 0% violation ở $s = 0.50\text{ m}$ nên được hiểu là privileged-map geometric sanity check, không phải perception-limited online autonomy. Vì vậy, đóng góp của track này không phải chứng minh bài toán planning hình học đã khó, mà là tạo chuẩn đo lặp lại để thấy khi map đã biết thì bottleneck tiếp theo nằm ở sensing, latency, risk và closed-loop validation.

Bảng 4 ghi rõ hyperparameter để tránh diễn giải nhầm về

Table 4. Cấu hình chính của planner baseline.

Planner	Cấu hình chính
A* grid	Resolution 0.10 m ; obstacle inflation $r + s = 0.70\text{ m}$; Euclidean heuristic.
RRT	1500 iterations; step 0.35 m ; goal sample rate 0.20; inflated radius 0.70 m .
RRT*	2500 iterations; step 0.30 m ; neighbor radius 0.75 m ; goal sample rate 0.15.
MPPI	512 rollouts; horizon 60; 10 iterations; temperature 1.0; noise $\sigma = 0.35\text{ m}$.

Table 5. Phân tích lại từ clearance đã lưu: violation khi đổi safety margin.

Phương pháp	$s = 0.30$	$s = 0.40$	$s = 0.50$	$s = 0.60$
Human	0.2067	0.3533	0.4467	0.5467
Straight-line	0.4400	0.5633	0.6533	0.7467
RRT*	0.0000	0.0000	0.0000	0.7400
MPPI	0.0000	0.0000	0.0033	0.1300

Table 6. Budget sweep planner trên 45 trial từ artifact reviewer.

Planner	Budget	Coll.	Viol.	Clear. (m)	Mean/P95 ms
RRT	300 iter	0.0000	0.0000	0.9031	2.0/2.7
RRT	900 iter	0.0000	0.0000	0.8913	1.6/2.4
RRT*	300 iter	0.0000	0.0000	0.8981	14.3/28.0
RRT*	900 iter	0.0000	0.0000	0.8721	92.8/187.3
MPPI	96 roll., 3 iter	0.0000	0.0000	0.9878	2.8/3.3
MPPI	384 roll., 8 iter	0.0000	0.0000	0.9878	8.7/9.7

fairness. Các planner dùng cùng obstacle radius $r = 0.20\text{ m}$ và safety margin $s = 0.50\text{ m}$, nhưng không dùng cùng compute budget tuyệt đối; vì vậy Bảng 6 bổ sung một sweep nhỏ để đọc trade-off runtime/quality thay vì chỉ so sánh một cấu hình đơn lẻ.

Bảng 5 không chạy lại planner mà chỉ recompute từ `min_boundary_clearance_m` trong artifact per-trial. Kết quả cho thấy kết luận về straight-line/human unsafe giữ ổn định khi đổi margin, nhưng ranking giữa planner an toàn nhạy với ngưỡng quá bảo thủ: ở $s = 0.60\text{ m}$, RRT* bị đánh violation cao do clearance trung bình thấp hơn MPPI. Với $n = 300$, Wilson 95% CI cho rate bằng 0 là khoảng $[0, 0.0126]$; riêng MPPI violation 0.0033 có CI xấp xỉ $[0.0006, 0.0186]$. Vì vậy các con số 0 nên được đọc là “không thấy lỗi trong 300 trial”, không phải bảo đảm xác suất lỗi bằng 0.

Bảng 6 chạy trên cùng 45 trial để trả lời câu hỏi fairness ở mức compute budget. Trong subset này tất cả cấu hình đều an toàn, nên khác biệt chính nằm ở runtime và độ mượt/clearance; RRT* tăng iteration làm compute tăng mạnh, còn MPPI trong prototype này ít đổi path vì seed path hình học đã đủ tốt cho các case được chọn. Runtime của RRT có thể không tăng đơn điệu theo iteration do early connection/shortcutting và nhiễu đo thời gian trong Python prototype. Bảng này chưa thay thế benchmark 300 trial, nhưng giúp giải thích vì sao không nên đọc compute time của RRT* như một controller tối ưu onboard.

B. Chất lượng quỹ đạo 3D

Ngoài bảng safety 2D ở Bảng 3, đề tài có thêm proxy chất lượng quỹ đạo 3D trên 16 trial có đủ thông tin height. Hình 2 minh họa một case ODA trial 345; path planner vẫn chạy trên footprint $x - z$, sau đó được lift theo height start-goal và resample theo arclength để đo length, smoothness và max-turn trong Bảng 7. Đây chưa phải kiểm tra dynamics đầy đủ của quadrotor, nhưng giúp loại đường đi quá gấp khúc trước khi

Table 7. Trajectory quality proxy 3D trên 16 ODA trials.

Phương pháp	Trials	Length (m)	Smooth.	Max turn	Compute (ms)
Human	16	6.8853	0.115282	1.727	0.0
RRT	16	5.8320	0.001332	0.248	1.9
RRT*	16	5.7257	0.000069	0.046	394.7
MPPI	16	5.7448	0.000094	0.077	72.0

đưa sang mô phỏng.

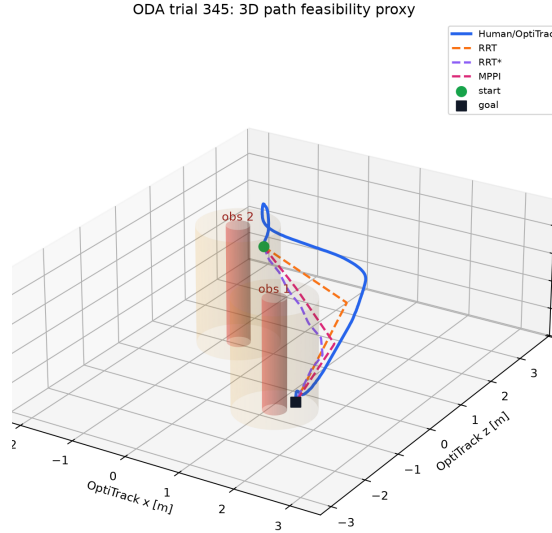


Fig. 2. Minh họa proxy feasibility 3D trên ODA trial 345: quỹ đạo Human/OptiTrack và các planner RRT, RRT*, MPPI quanh vật cản hình trụ.

Bảng 7 giải thích thêm trade-off trong planner: RRT rất nhanh nhưng max-turn lớn hơn; RRT*/MPPI cho đường mượt hơn rõ rệt, trong đó MPPI giữ compute time thấp hơn RRT* trong prototype Python. Human/OptiTrack có smoothness proxy cao vì quỹ đạo thật có nhiều đo và biến thiên theo thời gian, nên chỉ dùng làm reference thực tế, không dùng để kết luận human kém dynamics.

C. Latency và ODA replay

Latency được đánh giá ở hai mức. Mức thứ nhất đo sensor/front-end, tóm tắt ở Bảng 8: ODA RGB có tần số 5 fps nên riêng chu kỳ cảm biến đã là 200 ms; cached depth đổi sang occupancy chỉ mất khoảng 2.7 ms, nhưng nếu chạy Depth Anything V2 Small end-to-end thì front-end tăng lên 66.6 ms. Trên 50 ODA sequence với 2,584 frame, Depth Anything V2 Small đạt wall-time trung bình 64.0 ms/frame và inference 17.4 ms/frame; metric-depth indoor model nặng hơn, khoảng 190.5 ms/frame wall-time trên 15 sequence. Điều này cho thấy bottleneck online không chỉ nằm ở planner mà còn ở sensor period và depth frontend.

Trong Bảng 8, dòng “LiDAR bbox + ODA cached depth mux” vẫn bị giới hạn bởi timestamp ODA RGB/cached-depth 5 fps $\Rightarrow$ 200 ms; đây không phải claim về raw LiDAR 10 Hz. Stress-test LiDAR 10 Hz $\rightarrow$ ESDF $\rightarrow$ MPPI được tách riêng ở đoạn sau với sensor period 100 ms, nhằm đánh giá latency envelope nếu có nguồn point-cloud nhanh hơn ODA RGB.

Mức thứ hai dùng ODA replay thật: chọn các timestamp OptiTrack gần vật cản, cộng tổng delay vào thời gian hiện tại,

nội suy pose tại $t + T_{\text{delay}}$, rồi replanning từ pose đã bị trễ. Tổng cộng có 384 case chi tiết, tương ứng 128 case cho mỗi planner. Kết quả ở Bảng 9 cho thấy MPPI không collision trong replay nhưng còn 11.7% violation; RRT/RRT* nhanh hoặc mượt hơn ở vài chỉ số nhưng có collision do một số case replanning từ pose trễ thất bại. Như vậy, ODA-Bench không chỉ chấm path offline tĩnh mà còn kiểm tra được câu hỏi thực dụng: lệnh tránh vật cản có đến kịp không.

Một stress-test phụ với LiDAR 10 Hz $\rightarrow$ ESDF $\rightarrow$ MPPI cho thấy tổng delay từ 144.7 đến 321.4 ms. Ở 2 m/s, UAV đi thêm khoảng 0.41 m trước khi lệnh mới có hiệu lực; ở 5 m/s bắt đầu safety violation và 6 m/s có collision trong kiểm tra kinematic-delay. Sweep 144 case cho cùng mô hình latency cho thấy MPPI 3D-ESDF đạt safe rate 100% trong proxy này từ 1 đến 4 m/s, sau đó rơi mạnh ở mức 5–6 m/s. Thí nghiệm này dùng cùng công thức delay để đặt ngưỡng tốc độ hợp lý trước khi chạy simulator hoặc robot thật, không thay thế closed-loop dynamics validation.

D. Behavior cloning và risk prediction

Plain BC-MPPI và Filtered BC-MPPI dùng cùng policy MLP nhỏ, input là feature trạng thái cục bộ trong FEATURE_COLUMNS, output là local displacement/action 2D ($\Delta x, \Delta z$) tới waypoint kế tiếp trong policy dataset. Split vẫn theo trial ID. Bản Filtered BC chỉ giữ train samples có raw clearance trên ngưỡng quantile 0.60, tương ứng 1.6315 m và 4,116 train samples trong artifact hiện tại; mục tiêu là giảm việc học theo các đoạn rollout quá gần vật cản. Theo Bảng 11, Filtered BC giảm safety violation trên 45 test trials từ 0.3111 xuống 0.0667 so với Plain BC, trong khi collision rate trên ODA-Bench test giữ ở 0.0444. Điều này cho thấy lọc sample theo clearance có tác dụng giảm near-miss behavior, dù policy-learning claim vẫn phải giới hạn vì kết quả closed-loop chưa ổn định tuyệt đối.

Bảng 10 cho thấy Plain BC và Filtered BC khác nhau ở tập train được lọc, còn kiến trúc, loss, epoch, batch size và protocol đánh giá giữ giống nhau. Vì Filtered BC dùng tập train nhỏ và sạch hơn, kết quả nên được đọc là ablation về lọc dữ liệu theo clearance, chưa phải bằng chứng policy-learning tổng quát hơn mọi distribution.

Risk predictor trong Bảng 11 cho thấy MLP tốt hơn TTC ở chất lượng ranking và calibration tổng thể: TTC đạt PR-AUC 0.784, precision 0.428 và ECE 0.614, trong khi MLP nhỏ đạt PR-AUC 0.885, precision 0.598 và ECE 0.094. Tuy nhiên tại threshold đang báo cáo, TTC có FNR 0.000 còn MLP có FNR 0.008; vì vậy kết luận đúng là MLP ranking tốt hơn, không phải MLP luôn an toàn hơn TTC ở mọi operating point. Nếu dùng cho cảnh báo thật, threshold cần được chọn theo yêu cầu recall/precision hoặc cost-sensitive calibration.

Bảng 12 so sánh TTC và MLP tại operating point cùng recall. Khi giữ recall xấp xỉ 0.95 hoặc 0.99, MLP cho precision cao hơn TTC, trong khi FNR tương đương theo ngưỡng đã chọn. Điều này củng cố claim hẹp: MLP hữu ích hơn cho ranking/thresholding risk, còn lựa chọn threshold cuối vẫn phải theo chi phí bỏ sót và báo động giả.

Table 8. Sensor/front-end latency dùng ODA RGB/cached depth, 8 seeds mỗi tốc độ.

Pipeline	Sensor (ms)	Frontend (ms)	Safe speed	First unsafe	First coll.
ODA cached depth → grid	200.0	2.7	3.0	4.0	4.0
ODA Depth Anything → grid	200.0	66.6	3.0	3.0	4.0
LiDAR bbox + ODA cached depth mux	200.0	3.1	3.0	3.0	4.0

Table 9. ODA replay online feasibility theo planner.

Planner	Cases	Safe	Viol.	Coll.	Delay (ms)	Compute (ms)
MPPI	128	88.3	11.7	0.0	169.7	31.0
RRT	128	89.8	0.8	9.4	149.9	11.2
RRT*	128	88.3	0.8	10.9	181.6	42.9

Table 10. Cấu hình Plain BC-MPPI và Filtered BC-MPPI.

Hạng mục	Thiết lập
Input/output	8D local state → 2D displacement ($\Delta x, \Delta z$).
Architecture	MLP 8 → 64 → 64 → 2, ReLU hidden layers.
Loss/optimizer	MSE action loss; mini-batch gradient descent, batch 256.
Training	80 epochs; learning rate 5×10^{-4} ; same val/test split.
Rollout cap	Max 100 rollout steps; max step 0.1722 m .
Plain train set	10,290 train samples.
Filtered train set	Clearance quantile 0.60 = 1.6315 m ; 4,116 train samples.

Table 11. Kết quả learning/risk chính trên ODA-Bench.

Track	Metric 1	Metric 2	Metric 3	Ghi chú
Plain BC	viol. 0.3111	coll. 0.0444	clear. 1.3365	45 test trials
Filtered BC	viol. 0.0667	coll. 0.0444	clear. 1.3809	lọc sample gần vật cản
TTC risk	PR-AUC 0.784	FNR 0.000	pos. 0.3383	heuristic đơn giản
MLP risk	PR-AUC 0.885	FNR 0.008	prec. 0.598	ranking/calibration tốt hơn

Table 12. Risk operating point trên cùng test split.

Method	Target recall	Recall	Precision	FNR
TTC	0.95	0.9531	0.6031	0.0469
MLP	0.95	0.9504	0.7034	0.0496
TTC	0.99	0.9906	0.5044	0.0094
MLP	0.99	0.9906	0.6189	0.0094

E. PyBullet validation gate

Validation trong PyBullet được dùng như gate kiểm tra lại rank trước khi thử nghiệm thật hơn. Cấu hình máy và simulator được nêu ở Bảng 1; setting từ code gồm obstacle radius 0.20 m , safety distance 0.50 m , reaction delay 0.15 s , PyBullet altitude 1.0 m , control frequency 48Hz, simulation frequency 240Hz; RRT* dùng 900 iterations, step size 0.35 m , neighbor radius 0.75 m , còn MPPI dùng 192 rollouts, 5 iterations và horizon 50 steps. Kết quả 20 cases/method ở speed=1 trong Bảng 13 được báo cáo theo thứ tự success/violation/collision: RRT* đạt 75/10/15, MPPI đạt 65/18/17, Filtered BC đạt 69/15/16. Rank-transfer Spearman $\rho = 0.40$ cho thấy thứ tự offline và simulator có tương quan vừa phải trong validation set hiện tại; ODA-Bench vì vậy đóng vai trò lọc và phân tích ứng viên trước simulator/robot thật.

Table 13. PyBullet validation, 20 cases/method, speed=1.

Phương pháp	Success	Violation	Collision
RRT*	0.75	0.10	0.15
MPPI	0.65	0.18	0.17
Filtered BC	0.69	0.15	0.16

Table 14. Manifest artifact/reproducibility của báo cáo.

Hạng mục	Đường dẫn/trạng thái
Mốc code/reproduce	Commit bdb1050; toàn bộ bảng chính và hình trong báo cáo được sinh từ artifact đi kèm release bundle.
Report source	reports/uav_oda_report.tex; PDF sinh ra tại reports/uav_oda_report.pdf.
Bảng chính	outputs/tables/: planner, BC/risk, replay, PyBullet validation và các bảng reviewer sensitivity/CI/budget.
Hình/video	outputs/figures/ và outputs/videos/; video định tính chỉ dùng minh họa case, không thay thế metric.
PyBullet	Kết quả validation 20 cases/method được lưu trong artifact PyBullet và được tóm tắt ở Bảng 13.

IV. ĐÍNH KÈM MÃ NGUỒN VÀ BỘ CÀI CHUƠNG TRÌNH

Mã nguồn được nộp qua repository GitHub: https://github.com/thanhlamauto/UAV_AI. Hướng dẫn cài đặt và chạy lại nằm trong file README.md ở thư mục gốc của repository. Tóm tắt quy trình cài đặt là clone repository, tạo Python virtual environment, cài các gói trong requirements.txt, đặt ODA raw data vào data/raw/ODA_Dataset/, sau đó chạy các lệnh reproduce được ghi trong README để sinh lại bảng metric, hình và video trong outputs/.

Các bảng 5, 6 và 12 được sinh bằng cách phân tích lại artifact đã lưu. Các lệnh reproduce trong README là nguồn kiểm chứng chính; release bundle đi kèm bản nộp chứa bảng, hình và video được trích dẫn trong báo cáo.

V. ĐÁNH GIÁ HIỆU QUẢ

Table 15. Đánh giá theo các tiêu chí của báo cáo rút gọn.

Tiêu chí	Đánh giá
Độ khó	Kết hợp UAV indoor, log cảm biến thật, ground truth OptiTrack, metric an toàn, policy learning và simulator validation.
Tính mới	Biến ODA từ raw dataset thành benchmark/evaluator có task, split, label, metric và baseline tracks.
Hiệu quả	Giảm rủi ro thử nghiệm bằng cách loại unsafe candidate sớm; so sánh được planner/BC/risk trên cùng trial.
Mình chứng	300 trials, 14,700 policy-dataset samples, planner safety table, trajectory-quality proxy, ODA latency replay 384 cases, Filtered BC violation giảm mạnh, MLP risk PR-AUC cao hơn TTC, PyBullet validation 20 cases/method.
Giới hạn	Latency replay mới là emulation trên log; PyBullet rank transfer mới ở mức $\rho = 0.40$; Filtered BC collision 4.44% trên ODA-Bench test và 16% trong PyBullet validation, nên policy-learning claim cần scoped.

Tóm lại, các tiêu chí trong Bảng 15 cho thấy ODA-Bench tạo chuỗi raw log $\rightarrow$ offline benchmark $\rightarrow$ validation gate, giúp chọn cấu hình đáng thử tiếp bằng số liệu.

VI. THREATS TO VALIDITY

Có sáu giới hạn chính cần đọc kèm kết quả. Thứ nhất, benchmark chính dùng footprint $x - z$ và obstacle cylinder bán kính 0.20 m , nên chưa thay thế đánh giá dynamics đầy đủ của quadrotor với acceleration, jerk, yaw-rate và actuator constraints. Thứ hai, planner baseline dùng obstacle map dựng từ metadata ODA; đây là privileged-map setting để kiểm tra evaluator và planner geometry, chưa phải perception online hoàn chỉnh từ sensor noise. Thứ ba, các planner chưa được budget-match đầy đủ trên toàn bộ 300 trial; budget sweep hiện chạy trên 45 trial để minh họa trade-off. Thứ tư, human collision/violation được tính theo mô hình hình học và safety margin của evaluator, không đồng nghĩa tất cả case đó là va chạm vật lý trong log gốc. Thứ năm, PyBullet validation được báo cáo trên 20 cases/method ở speed = 1 m/s , với rank transfer $\rho = 0.40$ trong validation set này. Thứ sáu, timing được đo trên nhiều môi trường phần cứng khác nhau, nên chỉ nên dùng để so sánh tương đối trong từng track; chưa claim FPS hoặc runtime onboard tối ưu.

VII. KẾT LUẬN

Đề tài đã hoàn thành phiên bản ODA-Bench rút gọn với các thành phần chính: data adapter, evaluator, safety labels, planner baseline, latency-aware replay, Filtered BC, risk predictor và PyBullet validation gate. Các kết quả trong Bảng 3, 7, 8, 9, 11 và 13 cho thấy benchmark tách được unsafe straight-line/human-near-miss behavior, đo được trade-off safety/smoothness/compute, chỉ ra giới hạn tốc độ do ODA RGB/depth latency, Filtered BC cải thiện violation so với Plain BC, MLP risk ranking tốt hơn TTC, và PyBullet validation đạt

RRT* success 0.75, Filtered BC 0.69, MPPI 0.65. Kết luận phù hợp là ODA-Bench giúp lọc và phân tích candidate trước simulator/robot thật trong một protocol lặp lại được.

Hướng phát triển tiếp theo là mở rộng validation coverage, đặc biệt PyBullet 100–300 cases/method với speed sweep $1\text{--}4\text{ m/s}$ và delay sweep $0\text{--}300\text{ ms}$, thêm seed variance/confidence interval cho các planner stochastic, làm budget-matched curve trên toàn bộ test set, cải thiện closed-loop robustness của Filtered BC và thêm offline RL baselines. Claim cuối cùng nên giữ ở mức: benchmark construction + evaluator + tasks + baselines + PyBullet validation gate.

TÀI LIỆU THAM KHẢO

- [1] J. Dupeyroux, R. Dinaux, N. Wessendorp, and G. C. H. E. de Croon, “A Novel Obstacle Detection and Avoidance Dataset for Drones,” 2022. <https://doi.org/10.1145/3522784.3522786>.
- [2] J. Dupeyroux et al., “Obstacle Detection & Avoidance Dataset Repository.” https://github.com/JuSquare/ODA_Dataset.
- [3] H. Yu, G. C. H. E. de Croon, and C. De Wagter, “AvoidBench: A high-fidelity vision-based obstacle avoidance benchmarking suite for multi-rotors,” ICRA 2023. <https://doi.org/10.1109/ICRA48891.2023.10161097>.
- [4] Y. Song, S. Naji, E. Kaufmann, A. Loquercio, and D. Scaramuzza, “Flightmare: A Flexible Quadrotor Simulator,” CoRL 2020. <http://arxiv.org/abs/2009.00563>.
- [5] A. Loquercio, A. I. Maqueda, C. R. del-Blanco, and D. Scaramuzza, “DroNet: Learning to Fly by Driving,” IEEE RA-L, 2018. <https://rpg.ifi.uzh.ch/dronet.html>.
- [6] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine, “D4RL: Datasets for Deep Data-Driven Reinforcement Learning,” 2020. <https://arxiv.org/abs/2004.07219>.
- [7] S. Fujimoto and S. S. Gu, “A Minimalist Approach to Offline Reinforcement Learning,” 2021. <https://arxiv.org/abs/2106.06860>.
- [8] L. Chen et al., “Decision Transformer: Reinforcement Learning via Sequence Modeling,” 2021. <https://arxiv.org/abs/2106.01345>.
- [9] J. Panerati et al., “gym-pybullet-drones: PyBullet-based Gym for quadrotor reinforcement learning,” 2021. <https://github.com/utiasDSL/gym-pybullet-drones>.